

Ex.No.10 Use Ghidra to disassemble and analyze the malware code.

Project Objectives

- To provide a hands-on guide to disassembling and analyzing malware with Ghidra.
 - To enable readers to recognize common malware functionalities, such as persistence mechanisms, anti-analysis techniques, and network communications.
 - To equip users with the skills to interpret low-level assembly code and understand high-level behaviours.
-

GitHub Repository Components

A GitHub repository for this project could include:

1. README.md

- **Overview:** Brief description of Ghidra, its installation, and how it is used for malware analysis.
- **Requirements:** List of requirements, including Ghidra, a VM setup, and any sample binaries (preferably benign samples or detailed steps to safely acquire controlled malware samples).
- **Project Outline:** A breakdown of the tutorial steps, linking to various sections or files within the repository.

2. Tutorial Steps (Markdown or Jupyter Notebooks)

- **Step-by-step instructions, broken down into digestible sections, such as:**
 - **Environment Setup:** Instructions on setting up a virtualized and isolated environment.
 - **Initial Analysis:** How to load a binary into Ghidra, perform auto-analysis, and identify entry points.
 - **Function Analysis:** Methods for identifying and analyzing critical functions, decompiling code, and using Ghidra's cross-referencing features.
 - **String and Import Analysis:** How to locate and interpret strings and imports related to malicious behavior.
 - **Advanced Techniques:** Dynamic analysis, control flow graphs, custom scripting with Ghidra's Python or Java capabilities, and obfuscation detection.

3. Example Scripts (Python or Java)

- **Scripts to automate repetitive analysis tasks, such as:**
 - **Extracting and listing all string references.**

- Identifying network-related functions or common indicators of malware.
 - Labeling functions and data segments based on known patterns (e.g., file system manipulation, registry edits).
 - Script Examples: Python or Java code snippets demonstrating API usage in Ghidra (using Ghidra's Headless Analyzer, for example, for automation).
4. Sample Data (Benign Binary or Hex Dump)
- Due to safety and ethical concerns, include only safe, benign binaries or a hex dump example rather than live malware. This can allow users to learn and practice without the risk of executing malicious code.
 - Sample binaries designed to exhibit typical malware-like behaviors, such as:
 - Executing system calls.
 - Writing to files.
 - Sending network data.
 - Alternatively, a dummy binary generator script that creates binaries with similar structures to malware but without harmful payloads.
5. Documentation and Comments
- Include explanations of the code within each script, detailing what each function does.
 - Inline comments in disassembly views, providing readers with insights into assembly commands, control flow structures, and the decompiled code.
6. Report Template
- A reporting template for documenting findings, which could be in Markdown or PDF format. The template should cover:
 - Summary of Analysis: Brief description of the suspected malware.
 - Function Analysis: Summary of the key functions, their purpose, and the findings.
 - Behavioral Indicators: Summary of malware behaviors, such as persistence, file system manipulation, or network communications.
 - Recommendations: Suggestions on further actions, such as containment, eradication, or further analysis.

Example Repository Structure

plaintext

ghidra-malware-analysis-project/

├── README.md

├── docs/

```

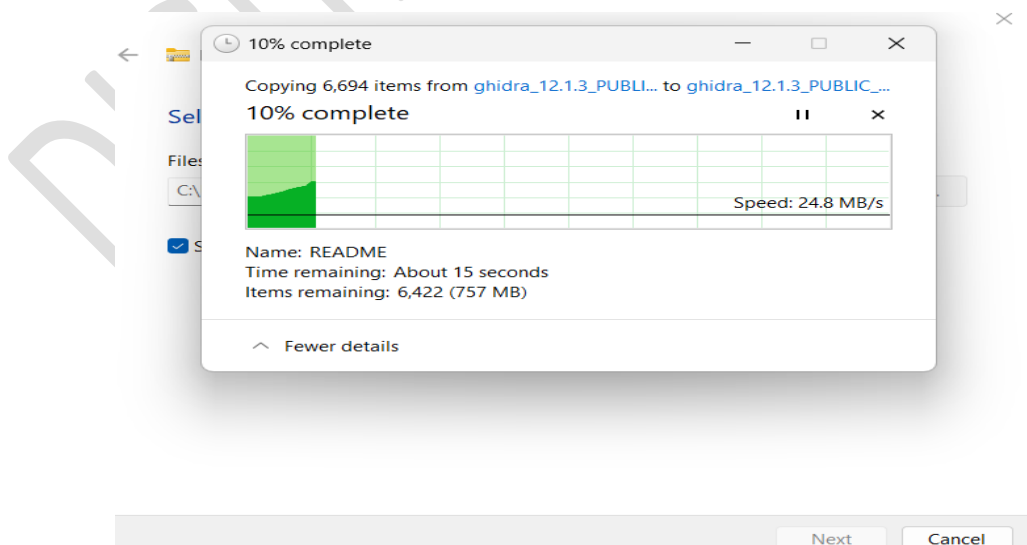
|   |— Tutorial_Step1.md
|   |— Tutorial_Step2.md
|   |— Tutorial_StepN.md
|— scripts/
|   |— extract_strings.py
|   |— network_analysis.py
|   |— label_functions.py
|— samples/
|   |— benign_sample1.bin
|   |— benign_sample2.hex
|— templates/
|   |— analysis_report_template.md
|— screenshots/
|   |— function_graph_example.png
|   |— decompiler_example.png

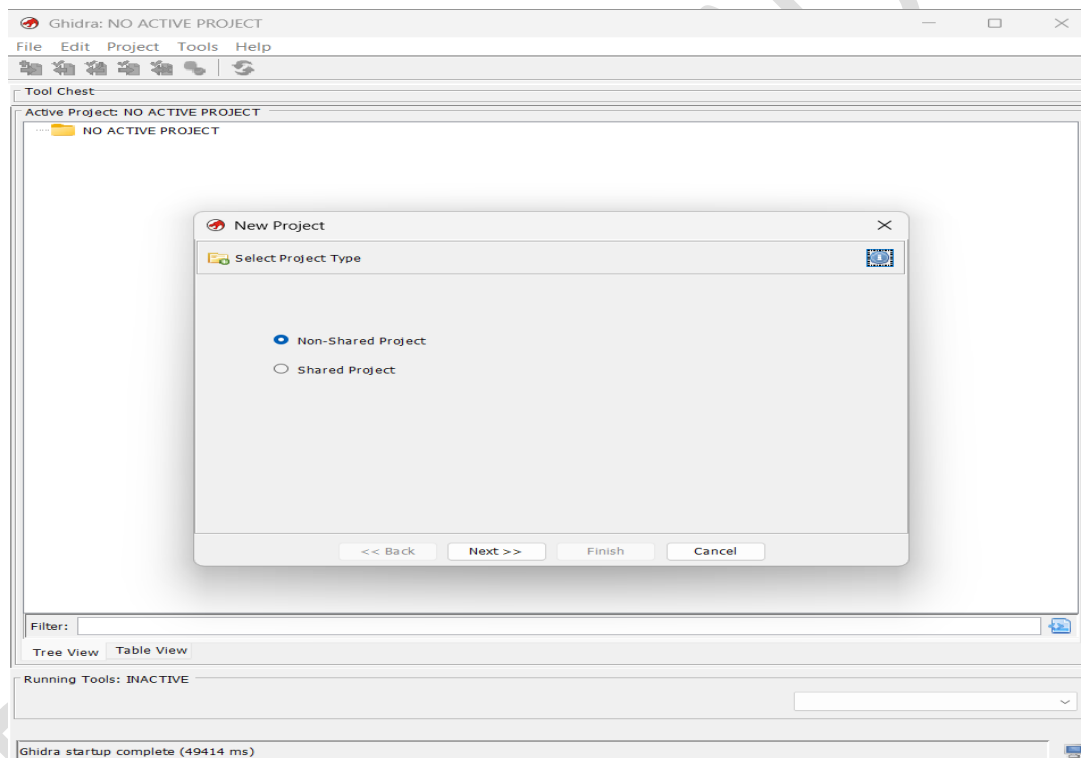
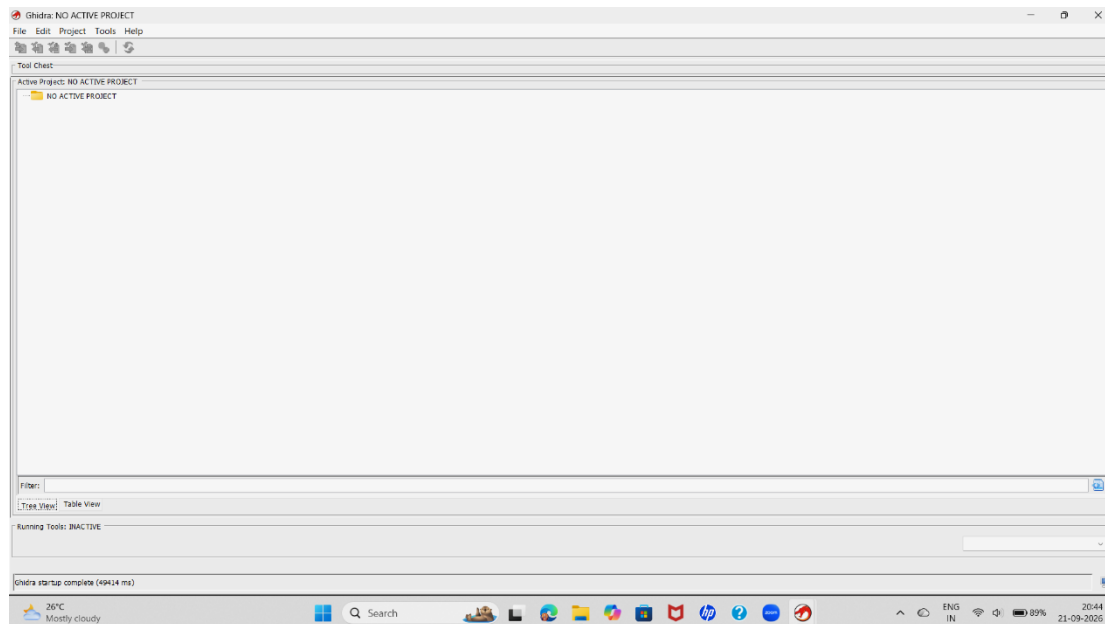
```

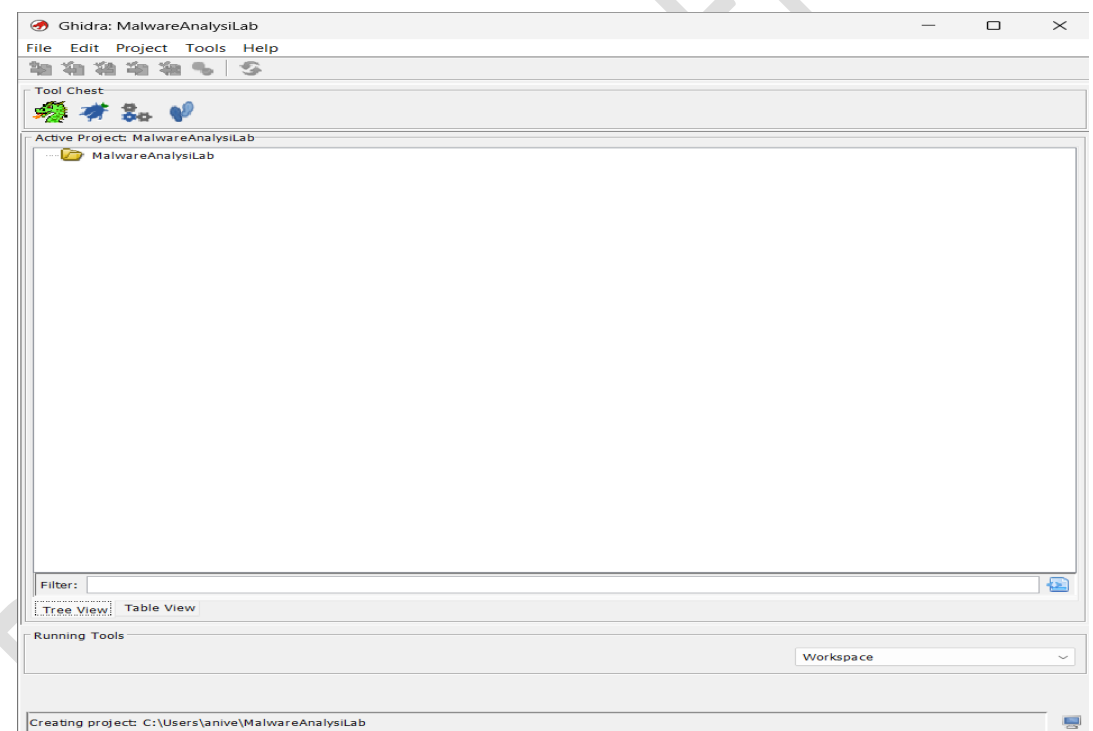
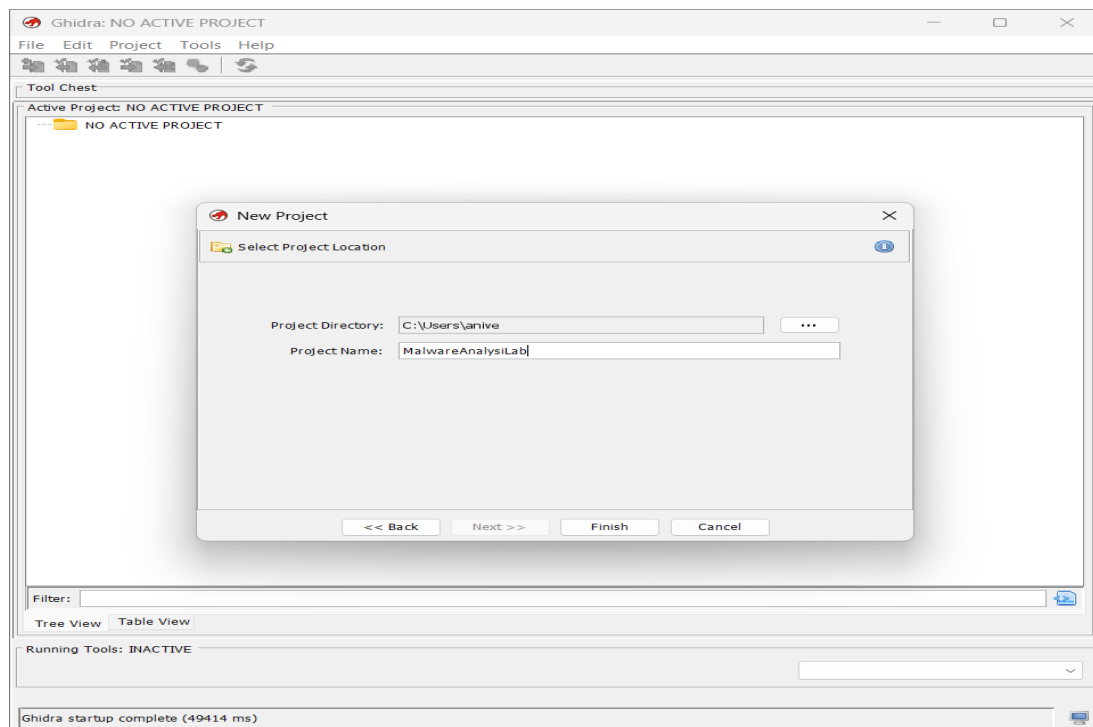
Project Development and Future Work

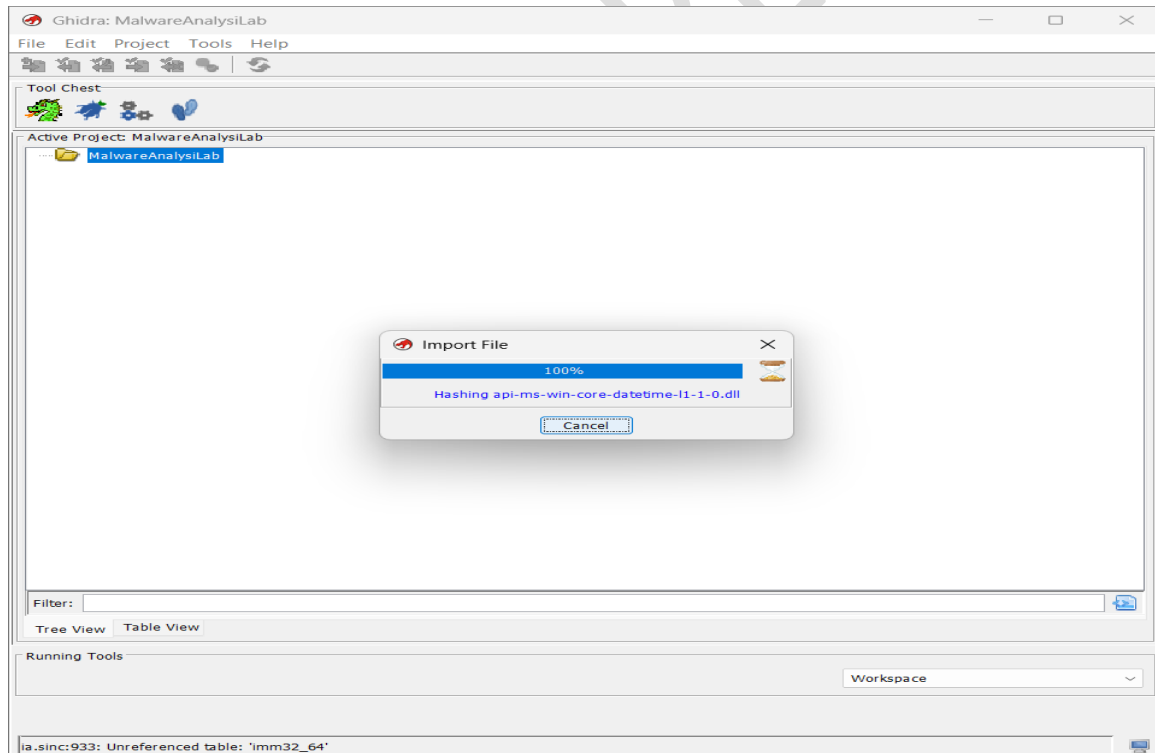
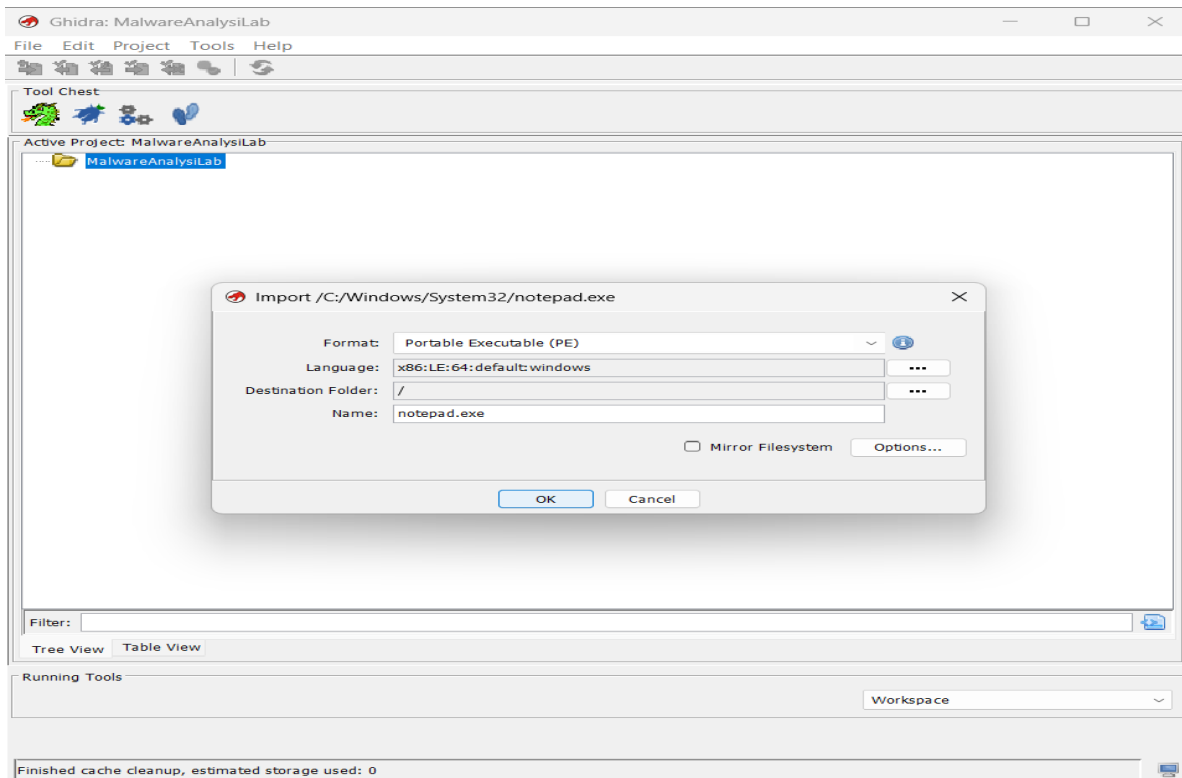
- **Expand Scripts:** Continue adding Ghidra scripts to automate complex analysis tasks.
- **Community Contributions:** Encourage community members to contribute additional benign samples, analysis scripts, and documentation improvements.
- **Advanced Topics:** Include guides for advanced techniques, such as unpacking obfuscated code, identifying encryption routines, and using the P-Code emulator for runtime analysis.

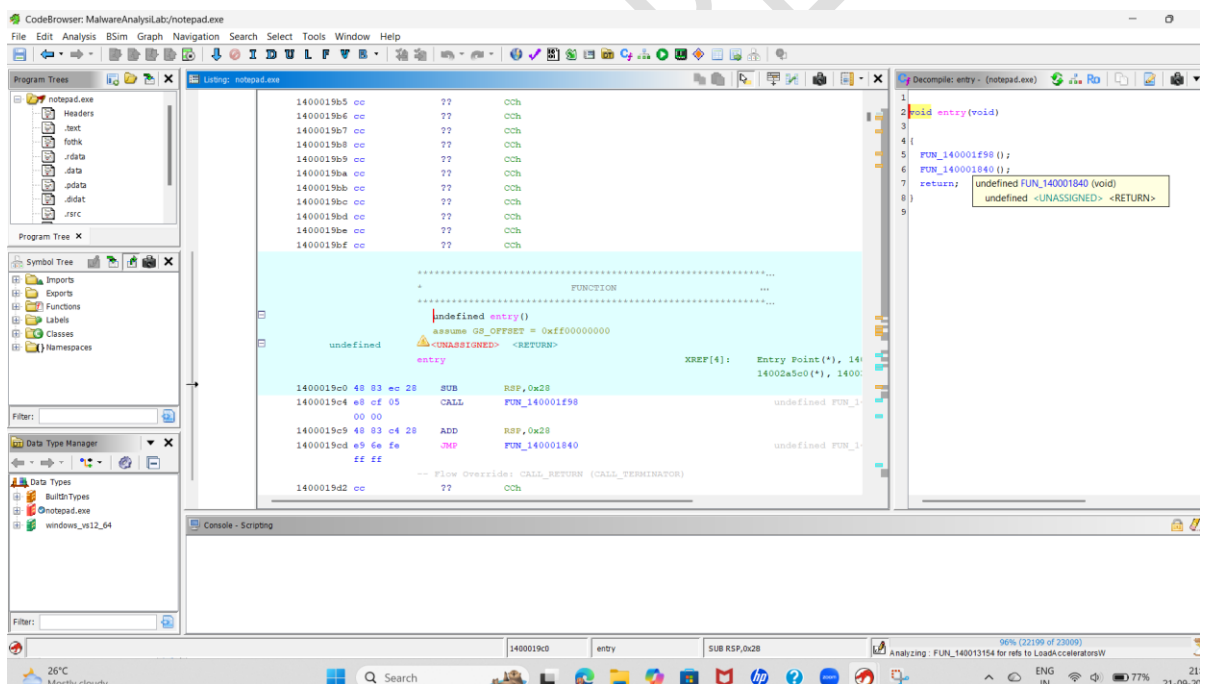
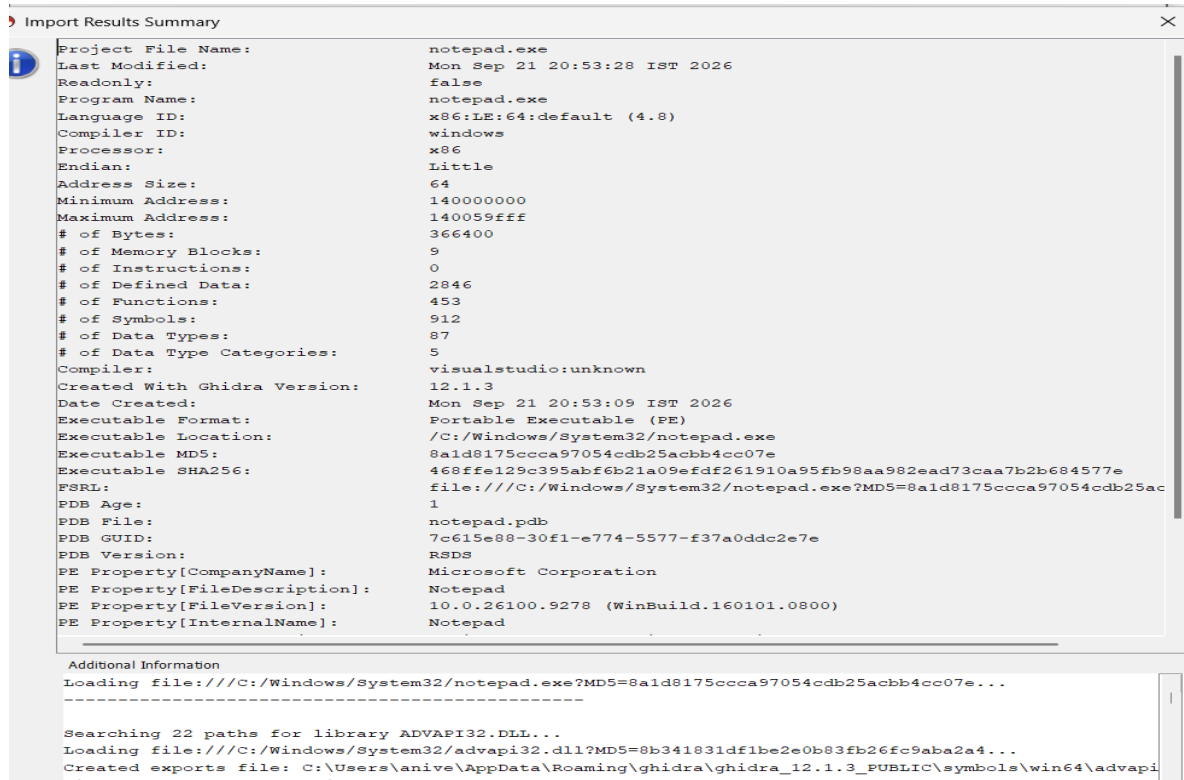
OUTPUTS:











Defined Strings [CodeBrowser: MalwareAnalysisLab/ntepad.exe]

Edit Help

Defined Strings - 682 items

Location	String Value	String Representation	Data Type
14002a03a	Microsoft.ZhengHei UI Bold	u"Microsoft.ZhengHei UI Bold"	unicode
14002a074	Microsoft.ZhengHei UI	u"Microsoft.ZhengHei UI"	unicode
14002a0a0	!QuerySystemInformation	"!QuerySystemInformation"	ds
14002a0c0	shell\\usshell\\accessory\\common\\edpapphelper\\edpapphelper.cpp	"shell\\usshell\\accessory\\common\\edpapphelper\\edpapphelper.cpp"	ds
14002a130	Windows.ApplicationModel.DataTransfer.Clipboard	u"Windows.ApplicationModel.DataTransfer.Clipboard"	unicode
14002a160	Windows.Security.EnterpriseData.ProtectionPolicyManager	u"Windows.Security.EnterpriseData.ProtectionPolicyManager"	unicode
14002a240	Windows.Security.EnterpriseData.FileProtectionManager	u"Windows.Security.EnterpriseData.FileProtectionManager"	unicode
14002a80f	Encoding5election	"Encoding5election"	ds
14002a821	SessionId	"SessionId"	ds
14002a82c	SequenceNumber	"SequenceNumber"	ds
14002a83b	Encoding	"Encoding"	ds
14002a855	PageSetupUpdated	"PageSetupUpdated"	ds
14002a866	SessionId	"SessionId"	ds
14002a871	SequenceNumber	"SequenceNumber"	ds
14002a880	HasHeaderOrFooter	"HasHeaderOrFooter"	ds
14002a8c3	ViewHelp	"ViewHelp"	ds
14002a8ac	SessionId	"SessionId"	ds
14002a8b7	SequenceNumber	"SequenceNumber"	ds
14002a8b6	TimeDataInvoked	"TimeDataInvoked"	ds
14002a8e6	SessionId	"SessionId"	ds
14002a8f1	SequenceNumber	"SequenceNumber"	ds
14002a900	BlogEntry	"BlogEntry"	ds
14002a91c	SearchingInvoked	"SearchingInvoked"	ds
14002a92a	SessionId	"SessionId"	ds
14002a939	SequenceNumber	"SequenceNumber"	ds
14002a948	entrypoint	"entrypoint"	ds
14002a955	hasQueryText	"hasQueryText"	ds
14002a972	FreshWindow	"FreshWindow"	ds
14002a97a	SessionId	"SessionId"	ds
14002a989	SequenceNumber	"SequenceNumber"	ds
14002a998	IsAdminMode	"IsAdminMode"	ds
14002a9b5	AppExit	"AppExit"	ds
14002a9bd	SessionId	"SessionId"	ds
14002a9c8	SequenceNumber	"SequenceNumber"	ds
14002a9d7	FileNewCount	"FileNewCount"	ds
14002a9e6	FileSaveCount	"FileSaveCount"	ds
14002a995	FileSaveAsCount	"FileSaveAsCount"	ds
14002a9a6	FilePrintCount	"FilePrintCount"	ds
14002a916	EditIndexCount	"EditIndexCount"	ds
14002a925	EditCutCount	"EditCutCount"	ds
14002a933	EditCopyCount	"EditCopyCount"	ds
14002a943	EditPasteCount	"EditPasteCount"	ds

Filter:

27°C Mostly cloudy

Search

ENG IN 74% 21:05 21-09-2026

Symbol Tree

Imports

- ADVAPI32.DLL
- API-MS-WIN-BASE-UTIL-L1-1-0.DLL
- API-MS-WIN-CORE-COM-L1-1-0.DLL
- API-MS-WIN-CORE-DATETIME-L1-1-0.DLL
- API-MS-WIN-CORE-DEBUG-L1-1-0.DLL
- API-MS-WIN-CORE-DELAYLOAD-L1-1-0.DLL
- API-MS-WIN-CORE-DELAYLOAD-L1-1-1.DLL
- API-MS-WIN-CORE-ERRORHANDLING-L1-1-0.DLL
- API-MS-WIN-CORE-FILE-L1-1-0.DLL

External Library Symbol - API-M

Filter: